

Projet CV-Forge :

Conception et réalisation d'un générateur de CV optimisé ATS avec Spring AI et Angular

Rapport d'architecture logicielle, choix de conception et implémentation technique

Fiche documentaire

Projet : CV-Forge (Générateur ATS-First)

Auteur : Florian Pépin

Version : 1.0

Date : 26 septembre 2026

Table des matières

1	Introduction	3
2	Partie I : Architecture et réalisation du backend	4
2.1	Choix de conception et justifications techniques	4
2.1.1	Axe 1 : Découplage strict entre Contenu structuré (IA) et Style graphique	4
2.1.2	Axe 2 : Mode d'inférence IA et sorties structurées (Structured Output) . .	4
2.1.3	Axe 3 : Stratégie de persistance et modélisation hybride	4
2.1.4	Axe 4 : Moteur de génération de CV et conformité ATS	6
2.1.5	Axe 5 : Spécificités culturelles et gestion de la photo selon le pays cible . .	6
2.1.6	Axe 6 : Modèle de session et gestion des utilisateurs	6
2.1.7	Axe 7 : Protection des coûts et limitation de débit (Rate-Limiting)	7
2.1.8	Axe 8 : Résilience, tolérance aux pannes et fallbacks IA	7
2.2	Modèle de données et schéma	8
2.3	Matrice de règles métier ATS et politique d'accès	8
2.4	Validation et tests automatisés du backend	8
3	Partie II : Architecture et réalisation du frontend	9
3.1	Choix de conception du client web	9
3.1.1	Axe 9 : Framework SPA réactif et composants Standalone	9
3.1.2	Axe 10 : Solution de mise en page et design visuel	9
3.1.3	Axe 11 : Prévisualisation dynamique et découplage visuel	9
3.1.4	Axe 12 : Séparation stricte des responsabilités et isolation des fichiers composants	10
3.1.5	Axe 13 : Ergonomie du Studio en écran partagé (Split-View)	10
3.2	Arborescence et architecture des composants	10
3.3	Écrans et parcours utilisateur	10
4	Partie III : Déploiement, gestion des secrets et conclusion	12
4.1	Orchestration conteneurisée et isolation des secrets	12
4.2	Conclusion et bilan de réalisation	12
A	Annexe 1 : Référentiel des End-points API	13
B	Annexe 2 : Référentiel des tests unitaires et d'intégration	14
B.1	Matrice des Tests Unitaires et de Rendu Documentaire	14
B.2	Matrice des Tests d'Intégration et des Contrats REST	14

1 Introduction

Le projet **CV-Forge** a pour objectif d'offrir une plateforme performante, intuitive et hautement qualifiée de génération de curriculum vitæ spécialement calibrés pour franchir avec succès les filtres des *Applicant Tracking Systems* (ATS). Les systèmes ATS représentent aujourd'hui le premier goulot d'étranglement du recrutement moderne : plus de 75 % des candidatures sont écartées avant même le moindre examen humain en raison de formats illisibles, de structures polychromes inadaptées ou d'un manque d'alignement sémantique avec la fiche de poste.

L'application est architecturée autour des composants suivants :

- **Backend RESTful & Orchestration IA** : Développé en Java 21 avec **Spring Boot 3.3**, exploitant **Spring AI** et les modèles de langage de dernière génération (**gpt-4o-mini** / **gpt-4o**) via des sorties rigoureusement typées (*Structured Outputs*). Le serveur prend en charge la transformation du texte brut, le matching d'offres et la protection par limitation de débit (**Bucket4j**).
- **Base de Données Hybride Relationnelle / Document** : Propulsée par **PostgreSQL 16**, combinant l'intégrité référentielle JPA/Hibernate pour l'agrégat racine et la flexibilité du stockage semi-structuré **JSONB** pour le contenu textuel et le paramétrage visuel.
- **Moteur de Rendu Documentaire ATS** : Moteur de génération vectoriel 100 % Java fondé sur **Thymeleaf** et **OpenHTMLtoPDF**, garantissant une disposition mono-colonne sémantique, des polices standardisées et un texte intégralement sélectionnable.
- **Frontend Single Page Application (SPA)** : Conçu avec **Angular 18/19** en architecture Standalone, exploitant les *Signals* réactifs et le système utilitaire **Tailwind CSS** pour une prévisualisation instantanée sans re-génération superflue.
- **Infrastructure & Conteneurisation** : Déploiement automatisé multi-conteneurs via **Docker Compose** avec isolation étanche des secrets et environnement de test en mémoire (**H2**).

2 Partie I : Architecture et réalisation du backend

2.1 Choix de conception et justifications techniques

2.1.1 Axe 1 : Découplage strict entre Contenu structuré (IA) et Style graphique

Question : Comment permettre à l'utilisateur de personnaliser l'apparence de son CV sans engendrer de surcoûts d'appels à l'IA ni de latence inutile ?

Options :

- ☒ Séparation architecturale totale entre **CvContent** (données textuelles) et **CvStyle** (cosmétique visuelle).
- ☐ Objet unique combinant données de contenu et paramètres visuels re-généré à chaque modification.
- ☐ Demander à l'IA de produire directement du code HTML ou CSS complet à chaque requête.

Pourquoi : L'intelligence artificielle est sollicitée pour son raisonnement textuel, sa capacité de synthèse et l'optimisation des mots-clés ATS. En revanche, modifier une couleur d'accentuation, la police de caractères, la densité d'interligne ou la disposition des puces est une opération 100 % déterministe. En isolant **CvContent** et **CvStyle**, l'utilisateur peut ajuster en temps réel l'apparence de son CV avec un temps de réponse instantané (< 10 ms) et un coût d'inférence nul.

2.1.2 Axe 2 : Mode d'inférence IA et sorties structurées (Structured Output)

Question : Comment garantir que la réponse produite par le modèle de langage soit immédiatement exploitable sans risque d'erreur de parsing ?

Options :

- ☒ Utilisation du mécanisme *Structured Output* de Spring AI avec sérialisation type-safe directe (`.entity(CvContent.class)`).
- ☐ Génération de texte libre Markdown puis extraction artisanale par expressions régulières (Regex).
- ☐ Demande informelle de JSON dans le prompt avec désérialisation manuelle via **ObjectMapper**.

Pourquoi : Le parsing manuel de texte libre ou de blocs de code JSON non contraints est notoirement fragile : les modèles de langage peuvent ajouter du texte conversationnel (« *Voici votre CV...* ») ou omettre des virgules. Le *Structured Output* impose un schéma JSON strict dès l'échantillonnage des jetons par le modèle. La désérialisation vers le DTO Java est garantie, éliminant tout échec de parsing à l'exécution.

2.1.3 Axe 3 : Stratégie de persistance et modélisation hybride

Question : Comment modéliser et stocker le CV en base de données pour concilier requêtes SQL rapides et flexibilité d'arborescence ?

Options :

- ☒ Modèle hybride PostgreSQL avec table racine **CvProfile** (UUID) et colonnes **JSONB** pour le contenu et le style.
- ☐ Modèle 100 % relationnel normalisé avec 6 tables jointes (`cv_experience`, `cv_bullets`, etc.).
- ☐ Base de données NoSQL orientée document (MongoDB) découplée de l'écosystème Spring Boot habituel.

Pourquoi : Un CV complet présente une structure hiérarchique polymorphe (liste variable d'expériences, bullets quantifiés, compétences catégorisées). Une normalisation relationnelle classique exigerait de multiples jointures, des opérations de cascade lourdes et des verrous complexes lors des mises à jour. Grâce au support natif du format JSONB par PostgreSQL et Hibernate 6 (`@JdbcTypeCode(SqlTypes.JSON)`), nous bénéficions du meilleur des deux mondes : requêtage direct par index et souplesse totale du schéma.

2.1.4 Axe 4 : Moteur de génération de CV et conformité ATS

Question : Quelle technologie de rendu adopter pour produire un fichier PDF parfaitement lisible par les robots de recrutement ATS ?

Options :

- ☒ Moteur de gabarit **Thymeleaf** associé à **OpenHTMLtoPDF** (100 % Java natif, flux vectoriel pur).
- ☐ Pilotage d'un navigateur headless externe (Playwright / Puppeteer / Chromium).
- ☐ Génération programmatique impérative de bas niveau avec Apache PDFBox ou iText.

Pourquoi : Les analyseurs ATS ont besoin d'un texte continu, sélectionnable, encodé en UTF-8 et exempt d'éléments graphiques complexes (colonnes flottantes, calques transparents ou images bitmap de texte) qui faussent la lecture de haut en bas. Thymeleaf permet de structurer un template HTML5 sémantique (**header**, **section**, **h1**, **ul**), tandis qu'OpenHTMLtoPDF le compile instantanément en Java sans aucune dépendance binaire externe lourde, garantissant une conformité ATS optimale.

2.1.5 Axe 5 : Spécificités culturelles et gestion de la photo selon le pays cible

Question : Comment traiter la présence de la photo de profil face aux divergences culturelles et aux normes juridiques internationales ?

Options :

- ☒ Règle métier automatisée dans l'agrégat : désactivation obligatoire de la photo pour les pays anglo-saxons (US, UK, CA, AU) et activation optionnelle pour l'Europe.
- ☐ Inclusion systématique de la photo sur tous les CV générés.
- ☐ Laisser l'utilisateur totalement libre sans avertissement ni ajustement automatique.

Pourquoi : Aux États-Unis, au Royaume-Uni et au Canada, les lois anti-discrimination interdisent formellement ou pénalisent lourdement la présence de photos sur les candidatures ; les logiciels ATS y rejettent souvent d'office les CV photographiés pour protéger l'employeur. À l'inverse, en France ou en Allemagne, la photo reste courante. L'agrégat racine **CvProfile** applique automatiquement cette règle dès la détection du code pays cible.

2.1.6 Axe 6 : Modèle de session et gestion des utilisateurs

Question : Comment permettre l'usage immédiat de la plateforme sans imposer la friction d'une création de compte obligatoire ?

Options :

- ☒ Service public avec identification par session anonyme persistante (**X-Session-Id** / **UUID**).
- ☐ Obligation de créer un compte avec email, mot de passe chiffré et confirmation par jeton **JWT**.
- ☐ Absence totale de persistance (flux purement éphémère sans possibilité de recharger son travail).

Pourquoi : Pour un générateur de CV en libre accès, exiger une inscription préalable fait fuir plus de 60 % des visiteurs. En attribuant un identifiant de session unique (**sessionId**) stocké dans le navigateur du client, l'utilisateur peut générer plusieurs CV, les recharger et les modifier à tout moment, tout en conservant une API totalement découplée et prête pour une authentification ultérieure.

2.1.7 Axe 7 : Protection des coûts et limitation de débit (Rate-Limiting)

Question : Comment prémunir le backend contre les abus, les attaques par déni de service et la surconsommation de l'API OpenAI ?

Options :

- ☒ Filtre de contrôle de flux basé sur l'algorithme du seau à jetons (**Bucket4j**) ciblé sur les routes IA par adresse IP.
- ☐ Limitation globale de bande passante au niveau du pare-feu réseau.
- ☐ Aucun contrôle de débit avec confiance accordée aux clients.

Pourquoi : Les appels aux modèles de langage représentent le coût opérationnel principal de la plateforme. Le filtre **RateLimitingFilter** intercepte les requêtes **POST** adressées à **/generate** et **/match**, accordant un crédit de 5 requêtes par minute par IP avant de renvoyer un code **429 Too Many Requests**, sans impacter la consultation ou l'exportation des CVs existants.

2.1.8 Axe 8 : Résilience, tolérance aux pannes et fallbacks IA

Question : Comment garantir que l'application reste utilisable et testable en environnement hors-ligne ou en cas d'indisponibilité du fournisseur LLM ?

Options :

- ☒ Détection automatique des clés démo/hors-ligne et bascule transparente vers un générateur de secours déterministe.
- ☐ Échec brutal avec code 500 dès que la connexion externe est indisponible.
- ☐ Blocage de l'application au démarrage en cas d'absence de variable d'environnement active.

Pourquoi : Lors des démonstrations, des sessions de développement local ou des tests d'intégration automatisés (**mvn test**), une dépendance stricte à une connexion Internet et à un compte OpenAI approvisionné est inacceptable. Le service **AiCvService** détecte les situations hors-ligne et fournit un contenu mocké riche et réaliste, garantissant une suite de tests 100 % verte en tout lieu.

2.2 Modèle de données et schéma

Le modèle de données de CV-Forge s'articule autour de l'agrégat racine **CvProfile** persistant en base relationnelle, hébergeant ses deux sous-systèmes documentaires au format JSONB :

```

+-----+
|                                     cv_profiles                                     |
+-----+
| id           : UUID [PK] |
| session_id   : VARCHAR(255) [INDEX] |
| user_email   : VARCHAR(255) |
| target_country : VARCHAR(10) (ex: 'FR', 'US', 'GB') |
| photo_url    : VARCHAR(500) (ex: '/api/v1/cv/photos/..') |
| created_at   : TIMESTAMP WITH TIME ZONE |
| updated_at   : TIMESTAMP WITH TIME ZONE |
+-----+
| content      : JSONB (Structure CvContent) |
|   +-- contactInfo : { fullName, email, phone, location, linkedinUrl } |
|   +-- summary     : "Profil professionnel oriente impact..." |
|   +-- experiences[] : [ { title, company, dates, bullets[] } ] |
|   +-- educations[] : [ { degree, institution, graduationDate } ] |
|   +-- skills[]    : [ { categoryName, skills[] } ] |
|   \-- matchResult : { matchScore, matchedKeywords, missingKeywords } |
+-----+
| style        : JSONB (Structure CvStyle) |
|   +-- primaryColor : VARCHAR (ex: '#1a365d') |
|   +-- fontFamily   : VARCHAR (ex: 'Helvetica', 'Times New Roman') |
|   +-- density      : ENUM ('COMPACT', 'NORMAL', 'SPACIOUS') |
|   +-- bulletStyle  : ENUM ('DISC', 'CIRCLE', 'SQUARE', 'HYPHEN') |
|   +-- sectionOrder : ['SUMMARY', 'EXPERIENCE', 'EDUCATION', 'SKILLS'] |
|   \-- showPhoto    : BOOLEAN (regle pays automatique) |
+-----+

```

2.3 Matrice de règles métier ATS et politique d'accès

Zone géographique	Photo autorisée	Disposition	Rubriques clés	Recommandation ATS
États-Unis / Canada	× (Désactivée)	Mono-colonne	Expérience, Compétences	Zéro photo, pas d'état c
Royaume-Uni (UK)	× (Désactivée)	Mono-colonne	Summary, Work Experience	Neutralité stricte anti-b
France / Benelux	✓ (Optionnelle)	Mono-colonne	Profil, Parcours, Diplômes	Photo discrète et profes
Allemagne (DACH)	✓ (Optionnelle)	Mono-colonne	Beruflicher Werdegang	Structure très factuelle

TABLE 1 – Matrice des règles d'adaptation culturelle et de conformité ATS

2.4 Validation et tests automatisés du backend

Afin d'assurer une fiabilité absolue du moteur de rendu, de la sérialisation JSONB et de la conformité des flux d'API, une suite complète de 16 tests automatisés a été implémentée avec JUnit 5, Spring Boot Test et MockMvc :

Domaine	Type	Scénarios validés	Tests	Statut
Moteur PDF ATS	Unitaire / Rendu	Flux vectoriel valide (%PDF-), styles, densités	3	✓
Orchestration IA	Métier	Génération structurée, extraction mots-clés	2	✓
Cycle de vie CV	Intégration JPA	Découplage style/contenu, règle pays ATS	2	✓
Contrôleur REST	MockMvc HTTP	Contrats d'API, validation 400, upload photo	7	✓
Rate-Limiting	Filtre Bucket4j	Débit nominal, blocage 429 au dépassement	2	✓
Total des tests exécutés via mvn test			16	100 %

TABLE 2 – Synthèse concise des tests automatisés du backend

3 Partie II : Architecture et réalisation du frontend

3.1 Choix de conception du client web

3.1.1 Axe 9 : Framework SPA réactif et composants Standalone

Question : Quelle technologie frontend privilégier pour une interface fluide, modulaire et hautement réactive ?

Options :

- ☒ **Angular 18/19** avec architecture *Standalone* intégrale et *Signals*.
- ☐ React avec bibliothèque externe de gestion d'état (Redux).
- ☐ Page HTML statique avec jQuery et rechargements périodiques.

Pourquoi : Angular offre un cadre robuste avec injection de dépendances native, validation fine des formulaires réactifs et typage TypeScript strict. L'utilisation des *Signals* permet une réactivité à granularité fine : lorsqu'un paramètre cosmétique est modifié, seul le composant d'aperçu est redessiné sans cycle global de détection des changements.

3.1.2 Axe 10 : Solution de mise en page et design visuel

Question : Comment assurer une esthétique soignée tout en facilitant l'ajustement dynamique des styles ?

Options :

- ☒ **Tailwind CSS** (utilitaires atomiques modernes et sur-mesure).
- ☐ Bootstrap 5 avec composants figés et classes impératives.
- ☐ Feuille de style CSS pure manuscrite sans système utilitaire.

Pourquoi : Tailwind CSS permet de concevoir des composants d'une grande finesse visuelle (sélecteurs de couleurs, badges de score ATS, bascules de densité) tout en éliminant les fichiers CSS superflus grâce à la purge automatique au build.

3.1.3 Axe 11 : Prévisualisation dynamique et découplage visuel

Question : Comment offrir à l'utilisateur un retour visuel instantané lors des retouches graphiques ?

Options :

- ☒ Rendu dynamique du modèle **CvContent** combiné aux variables réactives de **CvStyle** dans le DOM du navigateur.
- ☐ Re-génération complète du PDF côté serveur à chaque clic avec affichage dans un iframe.
- ☐ Absence d'aperçu avant le téléchargement final du fichier PDF.

Pourquoi : Télécharger un PDF généré par le serveur à chaque clic sur une nuance de couleur introduirait une latence insupportable et saturerait le réseau. L'aperçu Angular reflète fidèlement la structure du template PDF en appliquant les variables CSS locales, réservant l'appel au serveur pour l'exportation finale.

3.1.4 Axe 12 : Séparation stricte des responsabilités et isolation des fichiers composants

Question : Comment organiser l'architecture interne des composants Angular pour assurer une maintenabilité irréprochable et proscrire la dette technique ?

Options :

- ☒ Séparation rigoureuse et systématique en triplets de fichiers distincts (`.component.ts`, `.component.html`, `.component.css`) pour chaque composant applicatif, proscrivant formellement l'*inline*.
- ☐ Modèle hybride autorisant les templates et styles *inline* dans le fichier TypeScript.
- ☐ Styles globaux uniques partagés sans aucune encapsulation de composant.

Pourquoi : L'entremêlement du code logique TypeScript, du balisage HTML et des règles de style CSS dans un unique fichier monolithique nuit gravement à la lisibilité et complique les revues de code. En imposant un triplet de fichiers dédié pour chaque brique de l'interface (Header, InitialForm, Studio, EditorPanel, PreviewSheet, AtsGauge), chaque responsabilité demeure parfaitement isolée, maintenable et testable unitairement.

3.1.5 Axe 13 : Ergonomie du Studio en écran partagé (Split-View)

Question : Comment concevoir l'espace de travail pour concilier édition granulaire et vision globale du CV ?

Options :

- ☒ Espace Studio en écran partagé (*Split-View*) associant un volet d'outils à gauche (onglets Contenu, Style, Audit ATS) et la feuille A4 temps réel à droite.
- ☐ Assistant séquentiel par étapes masquant l'aperçu durant les modifications.
- ☐ Formulaire unique de saisie avec redirection vers une page d'aperçu finale.

Pourquoi : L'élaboration d'un CV est un processus itératif où chaque ajout textuel ou ajustement de marge influe sur la pagination et l'équilibre visuel du document. Le Studio Split-View permet au candidat d'ajuster une puce ou de modifier une nuance tout en observant immédiatement le résultat sur la feuille A4 virtuelle, sans jamais perdre le contexte de son document.

3.2 Arborescence et architecture des composants

3.3 Écrans et parcours utilisateur

L'application guide le candidat au travers d'un flux linéaire pensé pour minimiser l'effort de saisie :

1. **Écran 1 : Saisie brute des informations (/)** : Le candidat colle son parcours professionnel brut, ses formations, ses coordonnées et optionnellement l'annonce ciblée.
2. **Écran 2 : Atelier d'optimisation et d'aperçu en Split-View** : Affichage en direct du CV structuré par l'IA. Le candidat peut retoucher le texte manuellement, visualiser son score de matching ATS (mots-clés trouvés et suggestions), uploader sa photo et personnaliser son thème visuel (couleurs, puces, marges) sans latence serveur.

Composant Angular	Triplets de fichiers	Rôle et responsabilité fonctionnelle
AppComponent	.ts, .html, .css	Orchestrateur racine commutant entre l'ingestion brute et le Studio.
HeaderComponent	.ts, .html, .css	En-tête globale, historique des CVs de session, export PDF officiel.
InitialFormComponent	.ts, .html, .css	Saisie brute, sélecteur de pays avec règle anti-biais, chargement démo.
StudioComponent	.ts, .html, .css	Conteneur d'atelier Split-View coordonnant l'éditeur et la prévisualisation.
EditorPanelComponent	.ts, .html, .css	Volet gauche à 3 onglets (Contenu, Design & Style, Audit de matching).
PreviewSheetComponent	.ts, .html, .css	Rendu feuille A4 haute fidélité avec zoom réactif et typographies ATS.
AtsGaugeComponent	.ts, .html, .css	Jauge SVG circulaire animée indiquant le score d'adéquation ATS.

TABLE 3 – Répertoire modulaire des composants de l'application Angular

3. **Écran 3 : Exportation certifiée ATS** : Téléchargement en un clic du PDF vectoriel compilé par le moteur Spring Boot prêt pour les plateformes d'embauche.

4 Partie III : Déploiement, gestion des secrets et conclusion

4.1 Orchestration conteneurisée et isolation des secrets

L'ensemble de l'infrastructure de développement et de production est orchestré à l'aide d'un fichier `docker-compose.yml` :

- `cvforge-postgres` : Conteneur PostgreSQL 16 Alpine avec volume persistant (`cvforge_pgdata`) et contrôle de santé automatisé (*healthcheck*).
- `cvforge-backend` : Conteneur Spring Boot Java 21 optimisé pour un démarrage rapide.

Gestion rigoureuse des secrets :

- Aucun mot de passe ni clé d'API secrète n'est consigné dans le code source versionné.
- Un gabarit documenté `.env.example` est mis à disposition comme référence de configuration.
- Le fichier d'exclusion `.gitignore` bloque strictement tout commit de fichiers `.env` ou de répertoires de stockage local (`uploads/`).

4.2 Conclusion et bilan de réalisation

La plateforme **CV-Forge** constitue une solution complète, moderne et industrielle répondant aux exigences rigoureuses du recrutement contemporain :

- **Excellence fonctionnelle** : Découplage strict entre intelligence artificielle textuelle et personnalisation stylistique, évitant les gaspillages de ressources de calcul.
- **Conformité ATS sans compromis** : Modèle documentaire mono-colonne, respect des règles culturelles par pays (omission automatisée de photo pour US/UK/CA) et texte vectoriel 100 % sélectionnable.
- **Qualité logicielle démontrée** : Suite de tests automatisés exhaustive validant les contrats d'API, le rate-limiting, la persistance JSONB et la conformité du moteur de rendu PDF.

A Annexe 1 : Référentiel des End-points API

Méthode	Chemin (URI)	Accès	Rôle et description fonctionnelle
POST	/api/v1/cv/generate	Public / Session	Génération du CV structuré par l'IA à partir des données brutes saisies (soumis au rate-limiting).
GET	/api/v1/cv/{id}	Public / Session	Récupération du profil CV complet (contenu textuel, style et métadonnées).
GET	/api/v1/cv?sessionId=...	Public / Session	Consultation de la liste de tous les CVs rattachés à la session courante du candidat.
PUT	/api/v1/cv/{id}/content	Propriétaire	Modification manuelle du contenu textuel du CV sans solliciter le modèle d'IA.
PUT	/api/v1/cv/{id}/style	Propriétaire	Personnalisation cosmétique (couleurs, polices, densité, puces) sans appel IA.
POST	/api/v1/cv/{id}/match	Propriétaire	Analyse de matching ATS entre le CV et une nouvelle offre (score, mots-clés manquants).
POST	/api/v1/cv/{id}/photo	Propriétaire	Upload de la photo de profil avec validation de format (JPEG/PNG/WEBP ≤ 2 Mo).
GET	/api/v1/cv/photos/{name}	Public	Téléchargement et restitution directe de la ressource image de profil.
GET	/api/v1/cv/{id}/pdf	Public / Session	Génération et streaming du document PDF final compilé aux normes ATS.

TABLE 4 – Répertoire complet des endpoints de l'API REST CV-Forge

B Annexe 2 : Référentiel des tests unitaires et d'intégration

B.1 Matrice des Tests Unitaires et de Rendu Documentaire

Composant	Méthode de test (@Test)	Scénario et comportement attendu
PdfExportService	testRenderCvPdf_Success	Rendu d'un CV en document PDF vectoriel, vérification de la signature magique %PDF- et de la taille.
PdfExportService	testRenderCvPdf_WithCustomStyles	Validation du moteur de gabarit lors de l'application de densités (COMPACT) et polices variées.
PdfExportService	testRenderCvPdf_NullSafety	Contrôle de robustesse : levée d'une <code>IllegalArgumentException</code> si les données sont absentes.
AiCvService	testGenerateCvContent	Structuration nominale du texte brut en un objet <code>CvContent</code> doté d'expériences et de bullets d'action.
AiCvService	testAnalyzeJobMatch	Évaluation d'adéquation avec calcul objectif du score (0 à 100) et extraction des mots-clés de l'annonce.
RateLimitingFilter	testRateLimiting_Exceeded	Vérification du blocage HTTP 429 Too Many Requests dès le dépassement du quota fixé sur les routes IA.
RateLimitingFilter	testNonAiEndpoints_NotRateLimited	Garantie de non-blocage pour les requêtes de consultation ordinaires (méthodes GET).

TABLE 5 – Référentiel détaillé des tests unitaires et du moteur de rendu

B.2 Matrice des Tests d'Intégration et des Contrats REST

Composant	Méthode de test (@Test)	Scénario et comportement attendu
CvProfileService	testCreateAndLifecycle	Création persistance JPA, mise à jour isolée du style et du contenu sans déclenchement d'IA.
CvProfileService	testAtsCountryRule_PhotoSuppressedForUS	Règle métier ATS : masquage systématique de la photo lorsque le pays ciblé est anglo-saxon (US/UK).
CvProfileController	testGenerateCvEndpoint	Appel HTTP POST /generate vérifiant le code 201 Created et la conformité du payload JSON retourné.
CvProfileController	testGenerateCvEndpoint_ValidationFailure	Détection des contraintes Bean Validation et retour d'un ProblemDetail RFC 7807 (HTTP 400).
CvProfileController	testGetCvProfile	Interrogation GET /api/v1/cv/{id} validant la restitution conforme du profil persistant.
CvProfileController	testUpdateStyleEndpoint	Appel PUT validant la mise à jour exclusive des paramètres graphiques (primaryColor, etc.).
CvProfileController	testMatchEndpoint	Appel POST /match validant le recalibrage du score face à une annonce d'emploi transmise.
CvProfileController	testUploadPhotoEndpoint	Test de téléversement multipart validant le stockage sur disque et l'attribution de l'URI d'avatar.
CvProfileController	testDownloadPdfEndpoint	Téléchargement du PDF via l'API REST avec vérification de l'en-tête Content-Type: application/pdf.

TABLE 6 – Référentiel détaillé des tests d'intégration et des contrats d'API